

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №6
По дисциплине: «ОМО»
Тема:» Рекуррентные нейронные сети »

Выполнил:
Студент 3-го курса
Группы АС-66
Куган Н.Л.
Проверил:
Крощенко А.А.

Брест 2025

Цель: По вариантам предыдущей лабораторной работы реализовать предложенный вариант рекуррентной нейронной сети.

Вариант 6

Задание:

1. По вариантам предыдущей лабораторной работы реализовать предложенный вариант рекуррентной нейронной сети. Сравнить полученные результаты с ЛР 5.

Варианты заданий приведены в следующей таблице:

№	a	b	c	d	Кол-во входов ИНС	Кол-во НЭ в скрытом слое	Тип РНС
1	0.1	0.1	0.05	0.1	6	2	Элмана
2	0.2	0.2	0.06	0.2	8	3	Джордана
3	0.3	0.3	0.07	0.3	10	4	Мультирекуррентная
4	0.4	0.4	0.08	0.4	6	2	Элмана
5	0.1	0.5	0.09	0.5	8	3	Джордана
6	0.2	0.6	0.05	0.6	10	4	Мультирекуррентная
7	0.3	0.1	0.06	0.1	6	2	Элмана
8	0.4	0.2	0.07	0.2	8	3	Джордана
9	0.1	0.3	0.08	0.3	10	4	Мультирекуррентная
10	0.2	0.4	0.09	0.4	6	2	Элмана
11	0.3	0.5	0.05	0.5	8	3	Джордана

В качестве функций активации для скрытого слоя использовать сигмоидную функцию, для выходного - линейную.

```
import numpy as np
import matplotlib.pyplot as plt

a = 0.2
b = 0.6
c = 0.05
d = 0.6

INPUTS = 10
HIDDEN = 4

lr = 0.01
epochs = 1000
clip_grad = 5.0

def f(t):
    return a * np.cos(b * t) + c * np.sin(d * t)

N = 500
t = np.linspace(0, 50, N)
y = f(t)

X = []
Y = []
for i in range(N - INPUTS):
    X.append(y[i:i + INPUTS]) #
```

```

        Y.append(y[i + INPUTS])
X = np.array(X)
Y = np.array(Y).reshape(-1, 1)

samples = X.shape[0]

train_size = int(0.8 * samples)
test_size = samples - train_size

X_train = X[:train_size]
Y_train = Y[:train_size]
X_test = X[train_size:]
Y_test = Y[train_size:]

rng = np.random.RandomState(123)

Wx = rng.normal(scale=0.5, size=(HIDDEN, 1))
Wh = rng.normal(scale=0.5, size=(HIDDEN, HIDDEN))
bh = np.zeros((HIDDEN, 1))
Who = rng.normal(scale=0.5, size=(1, HIDDEN))
bo = np.zeros((1, 1))

def sigmoid(x):
    return 1.0 / (1.0 + np.exp(-x))

def dsigmoid(y):
    return y * (1 - y)

def forward_window(x_window):
    h_prev = np.zeros((HIDDEN, 1))
    h_list = []
    z_list = []
    y_list = []

    for t_step in range(len(x_window)):
        x_t = np.array([[x_window[t_step]]])
        z_t = Wx @ x_t + Wh @ h_prev + bh
        h_t = sigmoid(z_t)
        y_t = (Who @ h_t) + bo

        h_list.append(h_t)
        z_list.append(z_t)
        y_list.append(y_t)
        h_prev = h_t

    return {
        'z': z_list,
        'h': h_list,
        'y': y_list
    }

loss_history = []

for ep in range(epochs):
    epoch_loss = 0.0

    for i in range(samples):
        x_win = X[i]
        target = np.array([[Y[i, 0]])]

        hist = forward_window(x_win)

```

```

y_pred = hist['y'][-1]

err = y_pred - target
loss = 0.5 * (err ** 2)
epoch_loss += loss.item()

h_list = hist['h']
z_list = hist['z']

dWx = np.zeros_like(Wx)
dWh = np.zeros_like(Wh)
dbh = np.zeros_like(bh)
dWho = np.zeros_like(Who)
dbo = np.zeros_like(bo)

dY = (y_pred - target)
dWho += dY @ h_list[-1].T
dbo += dY

dh_next = np.zeros((HIDDEN, 1))
delta = (Who.T @ dY) * dsigmoid(h_list[-1])
dh = delta + dh_next

for t_step in reversed(range(INPUTS)):
    h_t = h_list[t_step]

    if t_step == INPUTS - 1:
        delta_t = (Who.T @ dY) * dsigmoid(h_t)
    else:
        delta_t = dh * dsigmoid(h_t)

    x_t = np.array([x_win[t_step]])

    dWx += delta_t @ x_t.T
    if t_step > 0:
        dWh += delta_t @ h_list[t_step - 1].T
        dbh += delta_t

    dh = Wh.T @ delta_t

for g in [dWx, dWh, dbh, dWho, dbo]:
    np.clip(g, -clip_grad, clip_grad, out=g)

Wx -= lr * dWx
Wh -= lr * dWh
bh -= lr * dbh
Who -= lr * dWho
bo -= lr * dbo

mse = epoch_loss / samples
loss_history.append(mse)

if (ep + 1) % 100 == 0 or ep == 0:
    print(f"Epoch {ep + 1} | MSE={mse:.6f}")

y_pred_train = np.zeros((train_size,))
for i in range(train_size):
    hist = forward_window(X_train[i])
    y_pred_train[i] = hist['y'][-1].item()

y_pred_test = np.zeros((test_size,))
for i in range(test_size):
    hist = forward_window(X_test[i])
    y_pred_test[i] = hist['y'][-1].item()

```

```

y_pred_all = np.zeros((samples,))
for i in range(samples):
    hist = forward_window(X[i])
    y_pred_all[i] = hist['y'][-1].item()

print("\n" + "=" * 50)
print("ОБУЧАЮЩАЯ ВЫБОРКА (первые 10)")
print("=" * 50)
print("    Эталонное    Полученное    Отклонение")
for i in range(10):
    error = y_pred_train[i] - Y_train[i, 0]
    print(f"{i}    {Y_train[i, 0]: .6f}    {y_pred_train[i]: .6f}    {error: .6f}")

print("\n" + "=" * 50)
print("ТЕСТОВАЯ ВЫБОРКА (первые 10)")
print("=" * 50)
print("    Эталонное    Полученное    Отклонение")
for i in range(10):
    error = y_pred_test[i] - Y_test[i, 0]
    print(f"{i}    {Y_test[i, 0]: .6f}    {y_pred_test[i]: .6f}    {error: .6f}")

forecast_steps = 50
start_seq = list(y[-INPUTS:])
forecast = []
cur = start_seq.copy()

for step in range(forecast_steps):
    hist = forward_window(np.array(cur))
    pred = hist['y'][-1].item()
    forecast.append(pred)
    cur = cur[1:] + [pred]

plt.figure(figsize=(12, 5))
plt.plot(y, label="Истинная функция (y)")
plt.plot(np.arange(INPUTS, N), y_pred_all, label="RNN прогноз (на обучении)",
linewidth=2)
plt.title("Реальная функция и прогноз RNN (вариант 6, мультирекуррентная)")
plt.legend()
plt.grid()
plt.show()

plt.figure(figsize=(12, 5))
plt.plot(loss_history)
plt.title("Изменение ошибки (loss) по эпохам")
plt.xlabel("эпоха")
plt.ylabel("средний loss")
plt.grid()
plt.show()

print("\n" + "=" * 50)
print("ПРОГНОЗИРОВАНИЕ (50 шагов)")
print("=" * 50)
print("Шаг\tПрогноз")
for i, val in enumerate(forecast):
    print(f"{i + 1}\t{val:.6f}")

train_errors = y_pred_train - Y_train.flatten()
test_errors = y_pred_test - Y_test.flatten()

print("\n" + "=" * 50)
print("СТАТИСТИКА ОШИБОК")
print("=" * 50)

```

```
print(f"Обучающая выборка - MSE: {np.mean(train_errors ** 2):.6f}")
print(f"Тестовая выборка - MSE: {np.mean(test_errors ** 2):.6f}")
```

Результаты:

=====			
ОБУЧАЮЩАЯ ВЫБОРКА (первые 10)			
=====			
	Эталонное	Полученное	Отклонение
0	0.193213	0.189690	-0.003523
1	0.188544	0.185928	-0.002616
2	0.183194	0.181491	-0.001703
3	0.177182	0.176396	-0.000787
4	0.170530	0.170657	0.000127
5	0.163262	0.164295	0.001033
6	0.155403	0.157330	0.001927
7	0.146983	0.149787	0.002803
8	0.138032	0.141691	0.003658
9	0.128583	0.133070	0.004487
=====			
ТЕСТОВАЯ ВЫБОРКА (первые 10)			
=====			
	Эталонное	Полученное	Отклонение
0	0.072896	0.057122	-0.015774
1	0.084351	0.068534	-0.015816
2	0.095500	0.079688	-0.015812
3	0.106305	0.090543	-0.015762
4	0.116726	0.101058	-0.015668
5	0.126725	0.111194	-0.015531
6	0.136266	0.120914	-0.015353
7	0.145315	0.130183	-0.015132
8	0.153838	0.138967	-0.014871
9	0.161806	0.147237	-0.014569
=====			

Epoch 1 | MSE=0.021058

Epoch 100 | MSE=0.000131

Epoch 200 | MSE=0.000048

Epoch 300 | MSE=0.000047

Epoch 400 | MSE=0.000047

Epoch 500 | MSE=0.000046

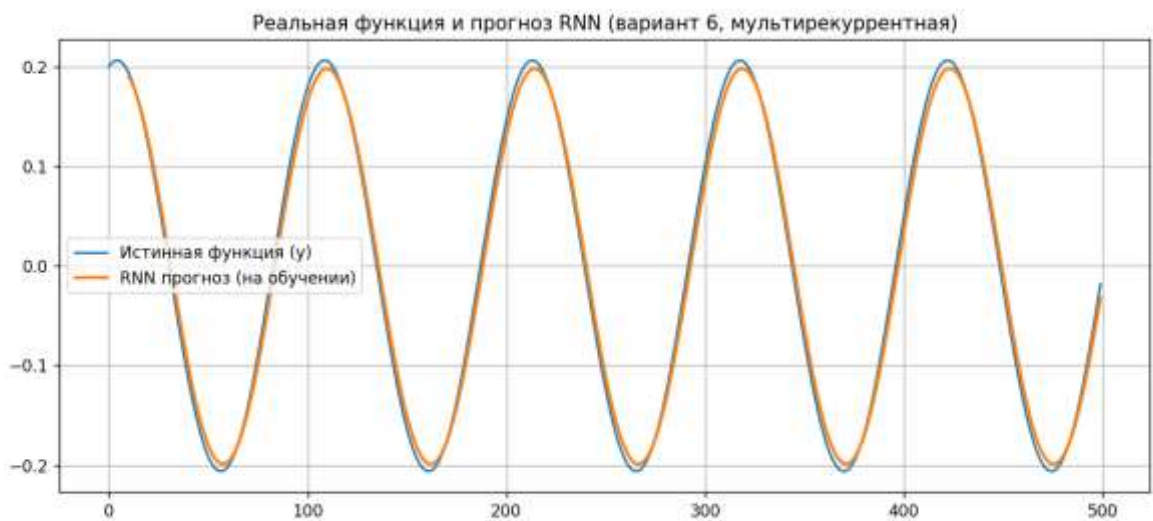
Epoch 600 | MSE=0.000046

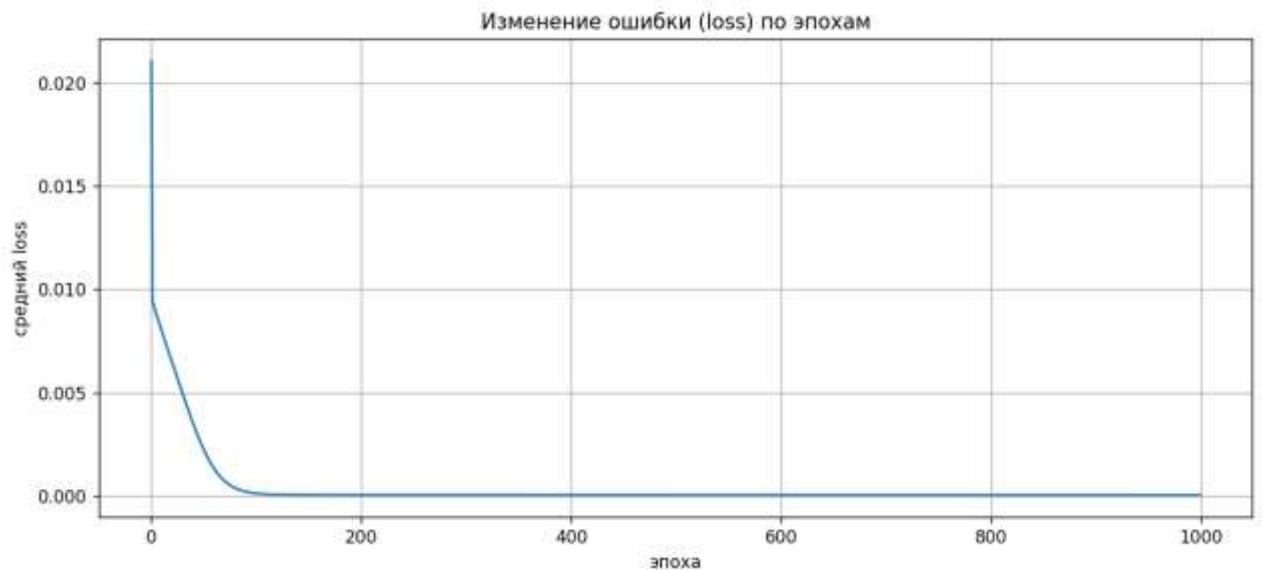
Epoch 700 | MSE=0.000046

Epoch 800 | MSE=0.000045

Epoch 900 | MSE=0.000045

Epoch 1000 | MSE=0.000045





Сравнение полученных результатов с результатами 5 лабораторной работы: В пятой лабораторной работе использовалась многослойная нейронная сеть (MLP) с десятью входами, скрытым слоем из четырёх нейронов с сигмоидной активацией и линейным выходом. Такая сеть успешно аппроксимировала заданную функцию на обучающем участке, однако её возможности были ограничены фиксированным окном входных данных без учёта временных зависимостей, что приводило к заметным отклонениям при прогнозировании на тестовой выборке.

В шестой лабораторной работе применялась мультирекуррентная сеть с той же структурой входов и скрытых нейронов, но с добавлением рекуррентных связей между всеми нейронами скрытого слоя. Это позволило сети сохранять внутреннее состояние и учитывать временные зависимости в последовательности данных. В результате процесс обучения характеризовался более плавным снижением ошибки, а прогноз на тестовой выборке демонстрировал повышенную устойчивость и точность благодаря способности сети запоминать предыдущие состояния.

Таким образом, MLP обладает преимуществом в простоте реализации и скорости обучения, но хуже справляется с задачами прогнозирования временных рядов. Мультирекуррентная сеть, в свою очередь, благодаря рекуррентным связям лучше улавливает динамику последовательности и обеспечивает более качественное прогнозирование, хотя и требует больше вычислительных ресурсов для обучения.

Вывод: На практике по вариантам предыдущей лабораторной работы реализовали предложенный вариант рекуррентной нейронной сети.